How WebAssembly Compares with Docker

This comparison of WebAssembly with Docker uses both theory and hands-on tests. By running a simple program in both environments, we explore their performance, security, portability, and startup behaviour to understand how each fits into today's software development landscape.

With modern software demanding faster launch times, smaller resource usage, and secure execution, two technologies stand out -- Docker containers and WebAssembly (Wasm). Docker changed the way we build and ship applications by using lightweight, isolated environments called containers. On the other hand, WebAssembly — originally built for the web — has grown into a powerful, cross-platform technology that runs code quickly and securely.

Docker is a platform for developing, shipping, and running applications inside containers. These containers package an application and its dependencies together, providing consistency across environments. Containers use OS-level virtualisation and share the host OS kernel. Docker is widely used in DevOps, CI/CD pipelines, and cloud deployments for its ease of use and integration with orchestration tools like Kubernetes.

WebAssembly (Wasm) is a low-level binary instruction format designed for a stack-based virtual machine. It was initially developed to run code in web browsers with near-native performance, but it has since expanded far beyond that scope. With the introduction of standalone runtimes like Wasmtime, Wasmer, and support from Docker, Wasm is now a serious option for running lightweight, cross-platform workloads outside the browser.

Wasm allows developers to write code in high-level languages like C, C++, or Rust, and compile it into compact, fast, and secure *.wasm* binaries. These binaries can execute in a sandboxed environment — providing memory safety and system isolation by default — making them ideal for use in plugins, embedded systems, serverless functions, and edge computing.

Key differences between Docker and WebAssembly

Feature	Docker container	WebAssembly (Wasm)
Startup time	Slower (seconds)	Fast (milliseconds)
Binary size	Large (MBs)	Very small (KBs)
Security model	Namespace isolation	Memory-safe sandbox
Language support	All languages	Best for C, C++, Rust
Portability	OS-dependent	Cross-platform bytecode

Docker and Wasm: A comparison

Both Docker and Wasm offer mechanisms for packaging and running code, but they target different use cases. Docker is ideal for complex applications that need an OS environment, while Wasm excels in microservices, plugin systems, and edge computing where performance and security are critical. Comparing them helps developers make informed decisions on tool selection for specific tasks.

Use cases and limitations: Docker containers are suitable for backend APIs, databases, and full-stack services. Their rich ecosystem and support for all programming languages make them universal. However, they carry more overhead and startup latency. WebAssembly is ideal for running small, secure code snippets, such as in serverless functions, IoT devices, and embedded systems. Its current limitations include limited support for dynamic linking, multi-threading, and mature debugging tools.

***Security considerations*:** Security is a critical factor in choosing between Docker and Wasm. Docker containers rely on OS-level isolation, using namespaces and control groups to sandbox applications. While effective, they still share the host kernel, making kernel-level vulnerabilities a concern. In contrast, WebAssembly runs in a strongly sandboxed environment with memory safety by design. Wasm restricts